DESENVOLVIMENTO MOBILE · MÓDULO 17 · COMUM ÀS TRILHAS · NÍVEL AVANÇADO

Backend Próprio

Servidores web, projeto de APIs REST, autenticação com JWT, PostgreSQL, implementação em Dart (Shelf) e Kotlin (Ktor), OpenAPI, Docker, HTTPS e segurança de APIs

Prof. Dr. Francisco dos Santos Viana

São Luís – Maranhão – Brasil · 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Explicar o caminho de uma requisição do app até o banco de dados.
- Projetar uma API REST com recursos, métodos, códigos de status e erros padronizados.
- Implementar autenticação com senhas protegidas e JWT, e autorização por papéis.
- Construir a API do LabMonitor em Dart (Shelf) e reconhecer o equivalente em Kotlin (Ktor).
- Documentar o contrato com OpenAPI.
- Empacotar com Docker e publicar com HTTPS, aplicando as práticas de segurança de APIs.

Roteiro

Neste módulo

- Fundamentos: como funciona um servidor web
- A API em Dart com Shelf
- O mesmo backend em Kotlin (Ktor)
- Contrato com OpenAPI
- Contêineres e implantação
- Segurança e operação
- Fechamento

Fundamentos: como funciona um servidor web

O caminho de uma requisição



DNS

Sistema que traduz `api.labmonitor.dev` no endereço IP do servidor.

Proxy reverso

Recebe as conexões da internet, cuida do HTTPS e encaminha para a aplicação, que fica protegida na rede interna.

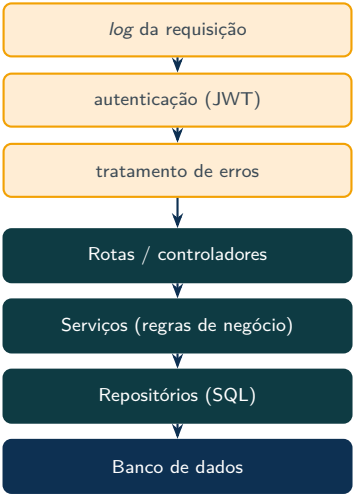
Servidor de aplicação

Processo que interpreta a requisição HTTP, executa a regra de negócio e responde, em geral com JSON.

Dica

Cada etapa pode falhar de um jeito diferente (nome não resolvido, certificado vencido, 502 do proxy, erro de SQL). Conhecer o caminho é o que permite diagnosticar.

Camadas e *middlewares* no servidor



Middleware

Função que envolve o tratamento de **todas** as requisições: pode registrar, validar, recusar ou modificar antes e depois do *handler*.

Mesmos princípios do app

Rotas só traduzem HTTP; serviços concentram as regras; repositórios falam com o banco. Separação de responsabilidades e inversão de dependência (Módulos 03 e 08) valem também aqui.

Analogia

Portaria de um prédio: identificação, registro e encaminhamento acontecem antes de chegar ao apartamento.

Projeto de APIs REST

REST

Estilo arquitetural descrito por Roy Fielding (2000): recursos identificados por URIs, manipulados por uma interface uniforme (métodos HTTP), com comunicação **sem estado** e respostas que podem ser guardadas em cache.

Requisição	Significado
`GET /v1/bancadas?livre=true`	listar com filtro
`GET /v1/bancadas/7`	uma bancada
`POST /v1/reservas`	criar (201 + `Location`)
`DELETE /v1/reservas/42`	cancelar (204)

Método	Seguro	Idempotente
GET	sim	sim
PUT	não	sim
DELETE	não	sim
POST	não	não
PATCH	não	não garantido

Idempotente

Repetir a requisição tem o mesmo efeito que fazê-la uma vez. Por isso é seguro repetir um `PUT` após uma falha de rede, mas não um `POST`.

Dica

Versione a API (`/v1`): apps antigos continuam funcionando quando a `/v2` chegar.

Erros padronizados e paginação

409 Conflict -- application/problem+json

```
{
  "type": "https://labmonitor.dev/erros/conflito",
  "title": "Horário indisponível",
  "status": 409,
  "detail": "A bancada B7 já está reservada das 14h às
      16h.",
  "instance": "/v1/reservas"
}
```

Problem Details (RFC 9457)

Formato padrão para erros de APIs HTTP: tipo, título, status e detalhe legível. O app mostra o `title` e decide pela combinação de `status` e `type`.

Paginação por cursor

```
GET /v1/leituras?bancada=7&limite=50&depois=9831
{
  "itens": [ ... 50 leituras ... ],
  "proximo": 9881
}
```

Por que paginar

Listas sem limite crescem com o tempo e derrubam app e servidor. O cursor ("depois do id 9831") é estável mesmo com inserções novas, ao contrário do deslocamento por página.

Autenticação: senhas e *tokens*

Autenticação e autorização

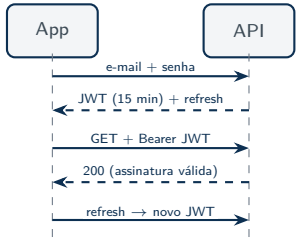
Autenticação: provar quem é. **Autorização**: decidir o que pode fazer.

Guardando senhas

O servidor guarda só o ***hash*** da senha, com algoritmo lento e **sal** (valor aleatório por usuário): Argon2id ou bcrypt. Nunca texto puro nem *hash* rápido (MD5, SHA-1).

Atenção

O JWT é **assinado**, não cifrado: nada de dados sensíveis no *payload*.



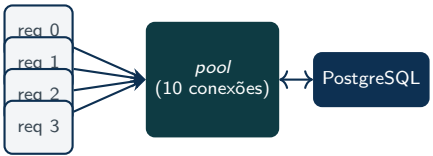
JWT (RFC 7519)

Cabeçalho, *payload* (`sub`, `exp`, papel) e assinatura, em Base64URL. O servidor confere a assinatura sem consultar o banco.

Banco de dados no servidor

Pool de conexões

Conexões abertas e reutilizadas. Abrir conexão é caro (rede, autenticação, TLS); o *pool* empresta uma pronta a cada requisição.



Práticas

- **Consultas parametrizadas** sempre (SQL *injection*, Módulo 09).
- **Transações** para operações que mudam várias tabelas.
- **Migrações** versionadas aplicadas na implantação.
- Usuário do banco com o **mínimo de privilégios**.
- Restrições no banco (`unique`, `check`, exclusão), como no Módulo 16.

A API em Dart com Shelf

Servidor, rotas e *pipeline*

bin/server.dart

```dart
Future<void> main() async {
  final db = Pool.withEndpoints([
    Endpoint(host: env('DB_HOST'), database: 'labmonitor',
        username: env('DB_USER'), password: env('DB_PASS')),
  ], settings: const PoolSettings(maxConnectionCount: 10,
      sslMode: SslMode.disable));            // rede interna
  final reservas = ReservaService(ReservaRepository(db));
  final rotas = Router()
    ..get('/saude', (Request r) => Response.ok('ok'))
    ..post('/v1/login', LoginHandler(db).call)
    ..get('/v1/bancadas', reservas.listarBancadas)
    ..post('/v1/reservas', reservas.criar)
    ..delete('/v1/reservas/<id|[0-9]+>', reservas.cancelar);
  final app = const Pipeline()
      .addMiddleware(logRequests())
      .addMiddleware(tratarErros())
      .addMiddleware(autenticar(env('JWT_SEGREDO')))
      .addHandler(rotas.call);
  await serve(app, InternetAddress.anyIPv4, 8080);
}
```

```
$ dart create -t server-shelf api
$ dart pub add shelf_router \
    postgres dart_jsonwebtoken bcrypt
$ dart run bin/server.dart
```

Shelf

Biblioteca do time do Dart para servidores HTTP: um *handler* recebe `Request` e devolve `Response`; *middlewares* se encaixam num `Pipeline`.

Dica

Configuração vem de variáveis de ambiente (`env`), nunca do código.

Repositório com SQL parametrizado

lib/reserva_repository.dart

```dart
class ReservaRepository {
  ReservaRepository(this.db);
  final Pool db;
  Future<List<Map<String, dynamic>>> bancadas() async {
    final r = await db.execute(
        'SELECT id, codigo, livre FROM bancadas ORDER BY codigo');
    return [for (final linha in r) linha.toColumnMap()];
  }
  Future<int> criar(String usuario, int bancada,
      DateTime inicio, DateTime fim) async {
    final r = await db.execute(
      Sql.named('INSERT INTO reservas '
          '(usuario_id, bancada_id, inicio, fim) '
          'VALUES (@u, @b, @i, @f) RETURNING id'),
      parameters: {'u': usuario, 'b': bancada,
                   'i': inicio, 'f': fim},
    );
    return r.first[0] as int;
  }
}
```

Parâmetros nomeados

O texto do SQL e os valores viajam separados; o PostgreSQL nunca interpreta o valor como código. É a defesa contra SQL *injection*.

E o conflito de horário?

Se a reserva sobrepõe outra, o banco lança o erro 23P01 (restrição de exclusão). O serviço o converte em `409 Conflict` no formato *Problem Details*.

Login e *middleware* de autenticação

lib/auth.dart

```
Middleware autenticar(String segredo) =>
    (Handler interno) => (Request req) {
  const publicas = {'saude', 'v1/login'};
  if (publicas.contains(req.url.path)) return interno(req);
  final h = req.headers['authorization'] ?? '';
  if (!h.startsWith('Bearer ')) {
    return Response.unauthorized('token ausente');
  }
  try {
    final jwt = JWT.verify(h.substring(7),
      SecretKey(segredo));
    return interno(req.change(context: {
      'usuario': jwt.payload['sub'],
      'papel': jwt.payload['papel'],
    }));
  } on JWTException {
    return Response.unauthorized('token inválido');
  }
};
```

No login, após conferir a senha

```
if (!BCrypt.checkpw(senha, usuario.hash)) {
  return Response.unauthorized('credenciais');
}
final token = JWT(
  {'sub': usuario.id, 'papel': usuario.papel},
).sign(SecretKey(segredo),
    expiresIn: const Duration(minutes: 15));
return Response.ok(jsonEncode({'token': token}),
    headers: {'content-type':
      'application/json'});
```

Dica

No cadastro: `BCrypt.hashpw(senha, BCrypt.gensalt())`. Mensagem de erro igual para e-mail e senha errados.

Autorização: o dono do recurso

lib/reserva_service.dart (cancelar)

```dart
Future<Response> cancelar(Request req, String id) async {
  final usuario = req.context['usuario'] as String;
  final papel = req.context['papel'] as String;
  final reserva = await repo.buscar(int.parse(id));
  if (reserva == null) return problema(404, 'Não encontrada');

  final dono = reserva.usuarioId == usuario;
  if (!dono && papel != 'tecnico') {
    return problema(403, 'Só o autor ou um técnico cancela');
  }
  await repo.apagar(reserva.id);
  return Response(204);
}
```

O defeito número 1 das APIs

BOLA (*Broken Object Level Authorization*): a API confere se o usuário está logado, mas não se o objeto pedido é dele. Basta trocar o 42 da URL para cancelar a reserva de outra pessoa.

RBAC

Controle de acesso por papéis: permissões ligadas a papéis (aluno, técnico, administrador), e usuários ligados a papéis.

O mesmo backend em Kotlin (Ktor)

Ktor: servidor, JSON e JWT

Application.kt

```
fun main() {
  embeddedServer(Netty, port = 8080) {
    install(ContentNegotiation) { json() }
    install(Authentication) {
      jwt("auth") {
        verifier(JWT.require(Algorithm.HMAC256(segredo))
          .build())
        validate { c ->
          c.payload.subject?.let { JWTPrincipal(c.payload) }
        }
      }
    }
    rotas(repo, servico)      // próximo slide
  }.start(wait = true)
}
```

Ktor

Framework da JetBrains, com *coroutines*; recursos entram como *plugins*.

Shelf (Dart)	Ktor (Kotlin)
`Router`	`routing { }`
`Pipeline` + `Middleware`	*plugins* e `intercept`
`Request.context`	`call.principal`
`postgres`	Exposed ou JDBC + HikariCP
`dart_jsonwebtoken`	`ktor-server-auth-jwt`
`dart compile exe`	*fat JAR* ou imagem nativa

Dica

App e servidor na mesma linguagem permitem compartilhar modelos e validações.

Ktor: rotas e autorização

Rotas.kt

```
fun Application.rotas(repo: Repo, servico: Servico) {
  routing {
    authenticate("auth") {
      get("/v1/bancadas") { call.respond(repo.bancadas()) }
      post("/v1/reservas") {
        val nova = call.receive<NovaReserva>()
        val uid = call.principal<JWTPrincipal>()!!.subject!!
        call.respond(HttpStatusCode.Created,
          servico.criar(uid, nova))
      }
      delete("/v1/reservas/{id}") {
        val id = call.parameters["id"]!!.toLong()
        val jwt = call.principal<JWTPrincipal>()!!
        val status = servico.cancelar(id, jwt.subject!!,
          jwt.getClaim("papel", String::class))
        call.respond(status)     // 204, 403 ou 404
      }
    }
  }
}
```

Mesma lógica do Shelf

Rotas protegidas ficam dentro de `authenticate`; o usuário vem do JWT; a regra de dono fica no serviço, que devolve um resultado traduzido em código HTTP.

Dica

`call.receive` desserializa o JSON com *kotlinx.serialization*; valide os campos antes de usar.

Contrato com OpenAPI

Descrevendo a API

openapi.yaml (trecho)

```yaml
openapi: 3.1.0
info: { title: LabMonitor API, version: 1.0.0 }
paths:
  /v1/reservas:
    post:
      summary: Cria uma reserva
      security: [{ bearer: [] }]
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/NovaReserva' }
      responses:
        '201': { description: Reserva criada }
        '401': { description: Sem autenticação }
        '409': { description: Horário indisponível }
```

OpenAPI

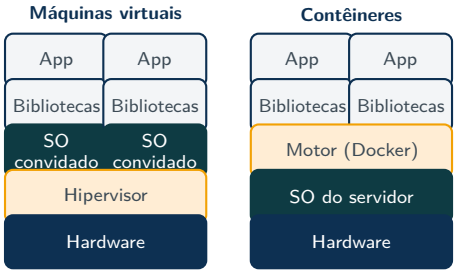
Especificação aberta para descrever APIs HTTP: rotas, parâmetros, corpos, respostas e segurança, num arquivo YAML ou JSON.

O que o contrato permite

- Documentação navegável (Swagger UI, Redoc).
- Gerar clientes para Dart e Kotlin.
- Testar se o servidor cumpre o contrato.
- Equipes de app e de servidor trabalharem em paralelo.

Contêineres e implantação

Máquinas virtuais e contêineres



Contêiner

Processo isolado que leva junto tudo de que precisa (bibliotecas, configuração), compartilhando o núcleo do sistema operacional. Sobe em segundos e roda igual no notebook e no servidor.

Termo	Significado
Imagem	modelo imutável (receita pronta)
Contêiner	imagem em execução
Registry	repositório de imagens (Docker Hub, GHCR)
Compose	vários contêineres descritos num arquivo

Empacotando a API

Dockerfile (estágios múltiplos)

```
FROM dart:stable AS build
WORKDIR /app
COPY pubspec.* ./
RUN dart pub get
COPY . .
RUN dart compile exe bin/server.dart \
    -o bin/server

FROM scratch
COPY --from=build /runtime/ /
COPY --from=build /app/bin/server /app/bin/
EXPOSE 8080
CMD ["/app/bin/server"]
```

Estágios múltiplos

O primeiro estágio compila; o final leva só o executável e o mínimo do sistema: imagem pequena, com pouca superfície de ataque.

compose.yaml

```
services:
  db:
    image: postgres:17
    environment:
      POSTGRES_DB: labmonitor
      POSTGRES_PASSWORD: ${DB_PASS}
    volumes: [dados:/var/lib/postgresql/data]
  api:
    build: .
    environment:
      DB_HOST: db
      DB_PASS: ${DB_PASS}
      JWT_SEGREDO: ${JWT_SEGREDO}
    depends_on: [db]
  proxy:
    image: caddy:2
    ports: ["80:80", "443:443"]
    volumes: [./Caddyfile:/etc/caddy/Caddyfile]
volumes:
  dados:
```

HTTPS e implantação

Caddyfile

```
api.labmonitor.dev {
  reverse_proxy api:8080
}
```

Certificado automático

O Caddy obtém e renova o certificado TLS com a Let's Encrypt pelo protocolo ACME: prova que controla o domínio e recebe um certificado gratuito válido por poucos meses, renovado sozinho.

```
$ docker compose up -d --build
$ docker compose logs -f api
```

Onde publicar	Características
Servidor da instituição	controle total; exige manutenção e rede liberada
VPS (máquina virtual)	barata e flexível; você cuida do sistema
PaaS de contêineres	Cloud Run, Fly.io, Render: escala sozinho; menos controle

Aplicação de doze fatores

Configuração no ambiente, *logs* na saída padrão, processos sem estado, banco como serviço anexado e ambientes de desenvolvimento e produção parecidos.

Atenção

Agende cópias de segurança do banco (`pg_dump`) e teste a restauração.

Segurança e operação

OWASP API Security Top 10 (2023)

Risco	Defesa no LabMonitor
API1 autorização por objeto (BOLA)	conferir o dono em toda rota com id
API2 autenticação quebrada	*hash* lento, JWT curto, limite de tentativas
API3 autorização por propriedade	aluno não altera o campo `papel`
API4 consumo irrestrito de recursos	paginação, limites de taxa e de tamanho
API5 autorização por função	rotas de técnico exigem papel técnico
API6 fluxos de negócio sensíveis	impedir reservas automáticas em massa
API7 falsificação de requisição no servidor	não buscar URLs enviadas pelo cliente
API8 configuração insegura	CORS restrito, sem mensagens de depuração
API9 inventário mal gerido	desligar versões e rotas antigas
API10 consumo inseguro de APIs de terceiros	validar o que outras APIs devolvem

Limite de taxa

Máximo de requisições por cliente num intervalo (ex.: 5 tentativas de login por minuto). Protege contra força bruta e abuso.

Operação

- Rota `/saude` para monitoramento.
- *Logs* estruturados (JSON), sem senhas nem dados pessoais.
- Segredos só em variáveis de ambiente.
- Dependências e imagens atualizadas.

Testando a API

test/reservas_test.dart

```dart
test('cancelar reserva de outra pessoa dá 403', () async {
  final app = montarApp(repo: RepoFake(reservas: [
    Reserva(id: 42, usuarioId: 'ana', bancadaId: 7),
  ]));
  final req = Request('DELETE',
      Uri.parse('http://teste/v1/reservas/42'),
      headers: {'authorization': 'Bearer
      ${tokenDe('bruno')}'});
  final resp = await app(req);
  expect(resp.statusCode, 403);
});
```

```
$ curl -i -X POST https://api.labmonitor.dev/v1/login \
    -H 'content-type: application/json' \
    -d '{"email":"ana@ifma.edu.br","senha":"..."}'
```

Camadas de teste

Unidade: serviços com repositórios *fake*.
Integração: API + PostgreSQL de teste em contêiner.
Contrato: respostas conferidas contra o `openapi.yaml`.
Manual: `curl`, Bruno ou Postman.

Dica

Teste os casos de segurança com o mesmo cuidado dos casos de sucesso: sem *token*, *token* vencido, recurso de outro usuário.

Fechamento

Resumo

Resumo

- Uma requisição passa por DNS, TLS, proxy reverso, servidor de aplicação e banco.
- REST organiza a API em recursos e métodos; erros seguem o *Problem Details*.
- Senhas com *hash* lento e sal; JWT curto para autenticar; papéis e dono para autorizar.
- Shelf (Dart) e Ktor (Kotlin) implementam as mesmas ideias: rotas, *middlewares*, camadas.
- OpenAPI documenta o contrato; Docker e Caddy entregam a API com HTTPS.
- BOLA é o risco número 1: confira sempre o dono do recurso.

Glossário (1/2)

Termo	Significado
Proxy reverso	servidor que recebe as conexões externas, faz o TLS e encaminha à aplicação
Middleware	função que envolve todas as requisições (log, autenticação, erros)
REST	estilo de API com recursos, métodos HTTP e comunicação sem estado
Idempotência	repetir a operação tem o mesmo efeito que fazê-la uma vez
Problem Details	formato padrão de erro em APIs HTTP (RFC 9457)
Sal	valor aleatório combinado à senha antes do *hash*
JWT	*token* assinado com cabeçalho, *payload* e assinatura
RBAC	controle de acesso baseado em papéis

Glossário (2/2)

Termo	Significado
BOLA	falha em verificar se o objeto pedido pertence ao usuário
***Pool* de conexões**	conexões abertas reutilizadas entre requisições
OpenAPI	especificação para descrever o contrato de APIs HTTP
Contêiner	processo isolado com suas dependências, sobre o núcleo do sistema
Imagem	modelo imutável a partir do qual contêineres são criados
ACME / Let's Encrypt	protocolo e autoridade que emitem certificados TLS gratuitos e automáticos
Limite de taxa	máximo de requisições por cliente num intervalo
Doze fatores	princípios para aplicações em nuvem (configuração no ambiente, processos sem estado…)

Questões de revisão

1. Descreva o caminho de uma requisição do app até o banco e um erro típico de cada etapa.
2. Por que um `POST` repetido após falha de rede pode criar duas reservas? Como evitar?
3. Por que senhas devem usar um *hash* lento com sal, e não SHA-256 puro?
4. O que o servidor verifica num JWT? Por que ele não deve conter dados sensíveis?
5. Explique BOLA com um exemplo e mostre a correção.
6. Qual a vantagem de um *pool* de conexões?
7. Que problema a imagem Docker de estágios múltiplos resolve?
8. Para que serve o arquivo OpenAPI num projeto com equipes de app e de servidor?

Exercícios

Exercício 1 — API

Implemente `GET /v1/leituras` com paginação por cursor e filtro por bancada e período.

Exercício 2 — Segurança

Escreva testes para os casos sem *token*, *token* vencido, reserva de outra pessoa e papel insuficiente.

Exercício 3 — Contrato

Escreva o `openapi.yaml` completo da API e gere a documentação navegável.

LabMonitor — Etapa 17

API própria em Dart (ou Ktor) com login, reservas e leituras, empacotada em Docker com PostgreSQL e Caddy; o app troca o repositório do Supabase pelo da API sem mudar a UI.

Para aprofundar

- Shelf: `https://pub.dev/packages/shelf` postgres: `https://pub.dev/packages/postgres`
- Ktor: `https://ktor.io/docs`
- OpenAPI: `https://spec.openapis.org/oas/latest.html`
- OWASP API Security Top 10: `https://owasp.org/API-Security`
- Docker: `https://docs.docker.com/get-started`
- Caddy: `https://caddyserver.com/docs` Doze fatores: `https://12factor.net/pt_br`

Respostas das questões de revisão

1. App → DNS (nome não resolvido) → TLS (certificado vencido) → proxy (502, aplicação fora do ar) → servidor (500, exceção) → banco (conexão recusada, restrição violada).
2. `POST` não é idempotente: a primeira chegou, mas a resposta se perdeu. Use uma chave de idempotência enviada pelo app ou restrições no banco que recusem a duplicata.
3. *Hashes* rápidos permitem testar bilhões de senhas por segundo; o algoritmo lento e o sal tornam o ataque caro e impedem tabelas pré-calculadas.
4. A assinatura (com o segredo ou a chave pública) e a validade (`exp`). O *payload* só é codificado em Base64URL: qualquer um o lê.
5. `DELETE /v1/reservas/42` apaga a reserva de outra pessoa porque só se verificou o login. Correção: comparar o dono da reserva com o usuário do *token* (ou exigir papel técnico).
6. Evita abrir uma conexão nova (rede, autenticação, TLS) a cada requisição e limita a carga sobre o banco.
7. Separa compilação e execução: a imagem final não leva SDK nem código-fonte, fica menor e mais segura.
8. É o contrato comum: gera documentação e clientes, permite trabalho em paralelo e testes de conformidade.

Fim do Módulo 17

Próximo: Módulo 18 (comum) — Engenharia avançada: modularização, desempenho, observabilidade e CI/CD.